CHAPTER 7

The Lambda Calculus

Every closure you have written in Rust, `|x| x + 1`, `|order| order.total()`, `move || report`, descends from a notation that Alonzo Church introduced in the 1930s to study what it means for a function to be computable. His **λ-calculus** has exactly three constructs: variables, functions and applications. There are no numbers, no booleans, no loops and no data structures, and yet it can express every computation a Rust program can. It is the smallest programming language there is, and nearly every idea in Part II of this book (types, type safety, the Curry–Howard correspondence) is first stated for it.

This chapter studies the λ-calculus *without* types. That may seem like an odd place to start a book about type theory, but it is the right one: to see what types buy you, you need to see the language they constrain. You will meet programs that run forever, functions that take themselves as arguments, and numbers built out of nothing but functions. You will also meet the operation at the heart of every programming language implementation, **substitution**, and the subtle bug, *variable capture*, that makes it harder than it looks. In Chapter 8 we add types and watch most of the strangeness disappear.

In this chapter you will learn:

- the syntax of λ-terms, free and bound variables, and α-equivalence;
- capture-avoiding substitution, why the naive version is wrong, and how de Bruijn indices remove the problem altogether;
- β-reduction, normal forms, the reduction strategies normal order, call-by-name and call-by-value, and the Church–Rosser theorem;
- divergence (Ω) and recursion without names (the Y and Z combinators);
- Church encodings of booleans, pairs, numerals and arithmetic, computed by the book's `lambda` crate;
- how Lean represents λ-terms, defines reduction as an inductive relation, proves facts about it, and why it refuses a normalizer that has no fuel.

7.1 Three constructs

The terms of the untyped λ-calculus are given by a grammar with three cases:

$$t ::= x \mid \lambda x.\, t \mid t\ t$$

- A **variable** x.
- An **abstraction** $\lambda x.\, t$: the function with parameter x and body t. In Rust you would write `|x| t`.

- An **application** $t\ u$: the function t applied to the argument u. In Rust, `t(u)`.

That is all. The notation needs two conventions to keep parentheses under control. Application associates to the **left**, so $f\,a\,b$ means $(f\,a)\,b$: apply f to a, then apply the result to b. And the body of a λ extends **as far to the right as possible**, so $\lambda x.\, x\,y$ means $\lambda x.\, (x\,y)$, not $(\lambda x.\, x)\,y$.

Every function takes exactly one argument. A function of two arguments is a function that returns a function: $\lambda x.\, \lambda y.\, x$ takes x and returns a function that takes y and returns x. This is called **currying** (after Haskell Curry, although the idea is older), and we often abbreviate $\lambda x.\, \lambda y.\, t$ as $\lambda x\, y.\, t$. In Rust the same function is `|x| move |y| x`.

Some terms we will use over and over:

Term	Name	Rust closure
$\lambda x.\, x$	identity, I	`|x| x`
$\lambda x.\, \lambda y.\, x$	constant, K	`|x| move |_y| x`
$\lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x)$	composition	`|f| move |g| move |x| g(f(x))`
$\lambda x.\, x\,x$	self-application, δ	none (see the end of the chapter)
$(\lambda x.\, x\,x)\,(\lambda x.\, x\,x)$	Ω	none

The last two rows are the first sign that the untyped calculus can do things Rust cannot. We will come back to them.

The `lambda` crate

This chapter's Rust code is the crate `lambda` (in `code/rust/crates/lambda`). Its central type is the abstract syntax of λ-terms, an `enum` with one variant per grammar case:

```rust
/// A term of the untyped λ-calculus, with variables as names:
/// `t ::= x | λx. t | t t`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    /// A variable `x`.
    Var(String),
    /// An abstraction `λx. t`: the function that sends `x` to `t`.
    Lam(String, Box<Term>),
    /// An application `t u`: the function `t` applied to the argument `u`.
    App(Box<Term>, Box<Term>),
}
```

A `Lam` owns its body and an `App` owns both of its parts, so a term is a tree; the `Box`es are there because a recursive `enum` must have a known size (Chapter 11 explains why). The crate also has a small parser, so that examples can be written as text, `parse("λf. λx. f (f x)")` (with `\` accepted for `λ`), and a printer that follows the two conventions above and prints the minimum number of parentheses. We will not discuss the parser; it is a routine piece of code.

7.2 Free and bound variables

In $\lambda x.\, x\,y$ the two variables play different roles. The x refers to the parameter of the enclosing λ: it is **bound**. The y refers to nothing inside the term: it is **free**, and its meaning must come from outside,

the way a global variable or a variable from an enclosing scope gets its meaning in Rust. The set of free variables is defined by recursion on the term:

$$\mathrm{FV}(x) = \{x\} \qquad \mathrm{FV}(\lambda x.\, t) = \mathrm{FV}(t) \setminus \{x\} \qquad \mathrm{FV}(t\ u) = \mathrm{FV}(t) \cup \mathrm{FV}(u)$$

The middle equation is the only interesting one: a λ removes its own parameter from the free variables of its body. The **scope** of a binder λx is its body, and an occurrence of x is bound by the *nearest* enclosing λx. So in $\lambda x.\, \lambda x.\, x$ the occurrence of x belongs to the inner λ, exactly as an inner `let x` shadows an outer one in Rust. A term with no free variables is **closed**; closed terms are complete programs, open terms are fragments.

The same name can even be free and bound in one term: in $x\ (\lambda x.\, x)$ the first x is free and the second is bound. That is legal, but confusing, and you will see shortly that it is also where the bugs live. In the crate, `free_vars` is a direct transcription of the three equations:

```
/// The free variables of a term: the variables that are not bound by an
/// enclosing `λ`.
pub fn free_vars(&self) -> BTreeSet<String> {
    match self {
        Term::Var(x) => BTreeSet::from([x.clone()]),
        Term::Lam(x, body) => {
            let mut fv = body.free_vars();
            fv.remove(x);
            fv
        }
        Term::App(t, u) => {
            let mut fv = t.free_vars();
            fv.extend(u.free_vars());
            fv
        }
    }
}
```

The free variables of a function body are something you already know well under another name. When you write `|y| x + y` in Rust, the variable `x` is free in the closure body, and the compiler must arrange for the closure to have access to it: it **captures** `x`. The captures of a Rust closure are exactly the free variables of its body. Chapter 8 develops this connection.

7.3 Substitution, and the capture problem

How does a λ-term compute? By applying functions to arguments, and applying $\lambda x.\, t$ to s means: take the body t and replace every free occurrence of the parameter x by s. We write this operation $t[x := s]$, "t with s for x". (Chapters 4 and 6 wrote the same operation $t[s/x]$; both notations are common.) It is what a compiler does when it inlines a function call, and what you do by hand when you compute $(x \mapsto x^2 + 1)(3) = 3^2 + 1$.

The naive definition

The obvious definition replaces occurrences of x and stops at a λx, because inside it x means something else:

- $x[x := s] = s$, and $y[x := s] = y$ when $y \neq x$;
- $(t\ u)[x := s] = t[x := s]\ u[x := s]$;

- $(\lambda x.\, t)[x := s] = \lambda x.\, t$ (the parameter x shadows our x);
- $(\lambda y.\, t)[x := s] = \lambda y.\, t[x := s]$ when $y \neq x$.

A first example works fine. Substituting the identity for x in $x\, y$:

$$(x\ y)[x := \lambda z.\, z] = x[x := \lambda z.\, z]\ \ y[x := \lambda z.\, z] = (\lambda z.\, z)\ y$$

Now a second example. Take the constant function $\lambda y.\, x$: whatever you give it, it returns x. Substitute the variable y for x:

$$(\lambda y.\, x)[x := y] = \lambda y.\, (x[x := y]) = \lambda y.\, y$$

Something went badly wrong. We started with a function that ignores its argument and returns x, and we asked for the same function returning y instead (the y from outside, whatever it is). We ended with the identity function, which returns its argument. The free y that we substituted in has been **captured** by the λy that happened to be in the way: it now refers to the parameter, not to the outside variable it was supposed to mean.

This is not a contrived corner case. It changes the answers of programs. The term $K\, y\, z = (\lambda x.\, \lambda y.\, x)\, y\, z$ should evaluate to y, because K returns its first argument. Here it is, computed by a reducer that uses the naive substitution, next to the crate's real reducer (the naive substitution is written out in `crates/lambda/examples/ch07_capture.rs`, where it is clearly marked as wrong):

```
/// Textual replacement of `x` by `s`, stopping only at a `λx` that shadows
/// `x`. This is WRONG: it does not check whether a `λ` captures a free
/// variable of `s`.
fn naive_subst(t: &Term, x: &str, s: &Term) -> Term {
    match t {
        Term::Var(y) if y == x => s.clone(),
        Term::Var(_) => t.clone(),
        Term::Lam(y, _) if y == x => t.clone(),
        Term::Lam(y, body) => lam(y, naive_subst(body, x, s)),
        Term::App(f, a) => app(naive_subst(f, x, s), naive_subst(a, x, s)),
    }
}
```

```
(λy. x)[x := y]
  naive:            λy. y
  capture-avoiding: λy'. y

(λx. λy. x) y z
  with naive substitution:           z
  with capture-avoiding substitution: y
```

The naive reducer computes $(\lambda x.\, \lambda y.\, x)\, y\, z \to (\lambda y.\, y)\, z \to z$: the first step captured y, and the second step then returned the wrong argument.

You have met this bug before, in a different costume. In C, a macro that declares a local variable `x` and multiplies it by its argument goes wrong when the caller passes an expression that mentions the caller's own `x`: after textual expansion, that `x` refers to the macro's local variable instead. Rust's `macro_rules!` is **hygienic**: the local variables a macro introduces live in their own syntactic context and cannot capture the caller's, which is exactly capture avoidance. The chapter's closure example, `code/rust/examples/ch07_rust_lambdas.rs`, checks it:

```
/// Multiplies its argument by a local `x`. The `x` inside the macro and an `x`
/// at the call site are different variables: `macro_rules!` renames the
/// macro's local bindings, exactly like capture-avoiding substitution.
macro_rules! double_it {
    ($e:expr) => {{
        let x = 2;
        x * $e
    }};
}
```

```
double_it!(x) with x = 10 is 20, not 2 * 2 = 4
```

Capture-avoiding substitution

The fix is to notice that the *name* of a bound variable does not matter. The function $\lambda y.\, x$ is the same function as $\lambda w.\, x$: both ignore their argument and return x. So before substituting under a binder that would capture something, we rename the binder to a fresh name. The last case of the definition splits in two:

- $(\lambda y.\, t)[x := s] = \lambda y.\, t[x := s]$ when $y \neq x$ and $y \notin \mathrm{FV}(s)$: no danger, go ahead;
- $(\lambda y.\, t)[x := s] = \lambda y'.\, (t[y := y'])[x := s]$ when $y \neq x$ and $y \in \mathrm{FV}(s)$, where y' is a **fresh** name, one that occurs neither in t nor in s.

Let us redo the example step by step. We want $(\lambda y.\, x)[x := y]$.

1. The term is an abstraction $\lambda y.\, t$ with $t = x$, and the binder y is different from x, the variable we substitute for.
2. Is y free in the replacement $s = y$? Yes. If we went under the binder, the y in s would be captured. So we take the second case.
3. Choose a fresh name: y' occurs nowhere, so it will do.
4. Rename the binder: $t[y := y'] = x[y := y'] = x$, giving $\lambda y'.\, x$. This is the same function as before.
5. Now substitute safely: $\lambda y'.\, x[x := y] = \lambda y'.\, y$.

The result, $\lambda y'.\, y$, is a constant function returning the outside y, as it should be, and $K\, y\, z$ now evaluates to y. In the crate, the two cases look like this:

```
/// Capture-avoiding substitution `self[x := s]`: replace the free
/// occurrences of `x` in `self` by `s`, renaming bound variables of `self`
/// when they would capture a free variable of `s`.
pub fn subst(&self, x: &str, s: &Term) -> Term {
    match self {
        Term::Var(y) if y == x => s.clone(),
        Term::Var(_) => self.clone(),
        Term::App(t, u) => app(t.subst(x, s), u.subst(x, s)),
        // `λx. body` binds x: there is no free x inside, nothing to do.
        Term::Lam(y, _) if y == x => self.clone(),
        Term::Lam(y, body) => {
            if s.free_vars().contains(y) && body.free_vars().contains(x) {
                // Going under `λy` would capture the free `y` of `s`.
                // Rename the bound `y` to a name used nowhere nearby.
                let mut avoid = s.free_vars();
                avoid.extend(body.all_names());
                avoid.insert(x.to_string());
                let fresh_y = fresh(y, &avoid);
                let renamed = body.subst(y, &Term::Var(fresh_y.clone()));
                lam(&fresh_y, renamed.subst(x, s))
            } else {
```

```
                lam(y, body.subst(x, s))
            }
        }
    }
}
```

The condition also checks that x actually occurs free in the body: if it does not, nothing will be substituted, and there is nothing to rename. Fresh names are made by adding primes until the name is unused:

```
/// A variant of `base` (with primes appended) that is not in `avoid`.
pub fn fresh(base: &str, avoid: &BTreeSet<String>) -> String {
    let mut name = format!("{base}'");
    while avoid.contains(&name) {
        name.push('\'');
    }
    name
}
```

α-equivalence

Renaming a bound variable, consistently and without capturing anything, does not change what a term means. Two terms that differ only in this way are called **α-equivalent**, written $t \underset{\alpha}{\equiv} u$: $\lambda x.\, x \underset{\alpha}{\equiv} \lambda y.\, y$ and $\lambda x.\, \lambda y.\, x\, y \underset{\alpha}{\equiv} \lambda a.\, \lambda b.\, a\, b$, but $\lambda x.\, \lambda y.\, x \not\underset{\alpha}{\equiv} \lambda x.\, \lambda y.\, y$ (the first returns its first argument, the second its second).

From now on we treat α-equivalent terms as *the same term*, as your IDE does when it offers "rename variable" (a refactoring that is safe exactly when the new name captures nothing). Capture-avoiding substitution is only defined up to α anyway, since it depends on which fresh name you pick. Later in the chapter we will see a representation in which α-equivalent terms are *literally* equal.

7.4 β-reduction

With substitution in hand, computation is a single rule. A **redex** (reducible expression) is an application whose function part is an abstraction, $(\lambda x.\, t)\ s$, and it **β-reduces** to $t[x := s]$. Since a redex can appear anywhere inside a term, we also need rules that allow a step inside the parts of an application and inside the body of a λ. In the notation of Chapter 3, the one-step relation $t \to t'$ is defined by four rules:

$$\frac{}{(\lambda x.\, t)\ s \to t[x := s]}\ \textsf{Beta} \qquad \frac{t \to t'}{t\ s \to t'\ s}\ \textsf{AppL}$$

$$\frac{s \to s'}{t\ s \to t\ s'}\ \textsf{AppR} \qquad \frac{t \to t'}{\lambda x.\, t \to \lambda x.\, t'}\ \textsf{Abs}$$

Rule Beta does the work; the other three only say *where* it may be applied. We write $t \overset{*}{\to} t'$ for "zero or more steps". Here is a complete computation:

$$(\lambda x.\, \lambda y.\, x)\ a\ b \to (\lambda y.\, a)\ b \to a$$

The first step contracts the redex $(\lambda x.\, \lambda y.\, x)\ a$ (inside the left part of the outer application, by rule AppL), substituting a for x. The second contracts $(\lambda y.\, a)\ b$, substituting b for y; since y does not occur in a, the argument b simply disappears.

Look at what substitution does to its argument, because it will matter for the rest of the book. In $(\lambda x.\, x\ x)\ s \to s\ s$ the argument s is **duplicated**; in $(\lambda y.\, a)\ b \to a$ it is **discarded**. These are the term-level versions of the structural rules of Chapter 6, contraction and weakening. The λ-calculus uses them freely. Rust does not: duplicating a `String` needs an explicit `.clone()`, and in Part V we will build a λ-calculus that, like Rust, refuses to duplicate.

A term that contains no redex at all is in **normal form**: $\lambda x.\, x$, $x\ (\lambda y.\, y)$ and $\lambda f.\, \lambda x.\, f\ (f\ x)$ are normal forms; $\lambda x.\, (\lambda y.\, y)\ x$ is not, because there is a redex under the λ. A normal form is the "answer" of a computation. Not every term has one:

$$\Omega = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \to (x\ x)[x := \lambda x.\, x\ x] = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) = \Omega$$

Ω reduces to itself in one step, and that is the only step it can take, so every reduction sequence from Ω is infinite. It is the λ-calculus version of `loop {}`.

7.5 Reduction strategies

A term can contain several redexes, and the rules do not say which one to contract first. A **reduction strategy** makes that choice. Three strategies matter:

- **Normal order**: always contract the **leftmost, outermost** redex, the one whose λ comes first when you read the term from the left, and reduce under λ too. Arguments are substituted unevaluated.
- **Call-by-name**: like normal order, but never reduce under a λ and never inside an argument. A function body is not evaluated until the function is called, and an argument is not evaluated until it is used.
- **Call-by-value**: before contracting $(\lambda x.\, t)\ s$, first reduce the argument s to a **value** (an abstraction); never reduce under a λ.

You know call-by-value very well: it is what Rust does. When you call `f(g(x))`, Rust evaluates `g(x)` first, completely, and only then calls `f`. And Rust never evaluates the body of a closure until the closure is called. The crate implements the three strategies as one `step` function:

```
/// Which redex to contract when there are several.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Strategy {
    /// Leftmost-outermost redex first, also under `λ`. Finds the normal form
    /// whenever there is one.
    NormalOrder,
    /// Leftmost-outermost, but never under `λ` and never inside an argument:
    /// arguments are passed unevaluated.
    CallByName,
    /// Arguments are reduced to values before the call, never under `λ`.
    /// This is what Rust does.
    CallByValue,
}
```

```rust
/// One step of β-reduction with the given strategy, or `None` if the strategy
/// finds no redex to contract.
pub fn step(t: &Term, strategy: Strategy) -> Option<Term> {
    match t {
        Term::Var(_) => None,
        Term::Lam(x, body) => match strategy {
            Strategy::NormalOrder => step(body, strategy).map(|b| lam(x, b)),
            Strategy::CallByName | Strategy::CallByValue => None,
        },
        Term::App(f, a) => {
            if let Term::Lam(x, body) = &**f {
                // A redex (λx. body) a. Call-by-value contracts it only once
                // the argument is a value.
                if strategy != Strategy::CallByValue || is_value(a) {
                    return Some(body.subst(x, a));
                }
                return step(a, strategy).map(|a2| app((**f).clone(), a2));
            }
            if let Some(f2) = step(f, strategy) {
                return Some(app(f2, (**a).clone()));
            }
            match strategy {
                Strategy::CallByName => None,
                _ => step(a, strategy).map(|a2| app((**f).clone(), a2)),
            }
        }
    }
}
```

Read the `App` case carefully. If the function part is an abstraction, we have a redex at the top; normal order and call-by-name contract it at once, call-by-value only once the argument is a value (otherwise it steps inside the argument). If the function part is not an abstraction, all three strategies try to step inside it first; then call-by-name stops, and the other two continue with the argument. Around `step`, a function `reduce` runs a strategy for at most a given number of steps, the **fuel**, and reports whether it finished:

```rust
/// The result of running a reducer with a step limit.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Reduction {
    /// The last term reached.
    pub term: Term,
    /// The number of steps taken.
    pub steps: usize,
    /// `true` if no step was possible at the end; `false` if the fuel ran out.
    pub finished: bool,
}

/// Reduces `t` with `strategy` until no step applies, or until `fuel` steps
/// have been taken.
pub fn reduce(t: &Term, strategy: Strategy, fuel: usize) -> Reduction {
    let mut term = t.clone();
    let mut steps = 0;
    while steps < fuel {
        match step(&term, strategy) {
            Some(next) => {
                term = next;
                steps += 1;
            }
            None => {
                return Reduction {
                    term,
                    steps,
                    finished: true,
                };
            }
```

```
        }
    }
    let finished = step(&term, strategy).is_none();
    Reduction {
        term,
        steps,
        finished,
    }
}
```

The fuel is not an implementation detail. A reducer for the untyped λ-calculus *cannot* know in advance whether a term has a normal form (that is the halting problem in disguise), so a reducer that wants to be sure of stopping must give up after some number of steps. We will meet the same idea in Lean, where it is not optional.

Strategies disagree about termination

The example `crates/lambda/examples/ch07_strategies.rs` runs the three strategies on a few terms. The first is a function that throws its argument away, applied to Ω:

```
(λx. λy. y) ((λx. x x) (λx. x x))
  normal order   λy. y   (1 step)
  call-by-name   λy. y   (1 step)
  call-by-value  out of fuel after 1000 steps
```

Normal order and call-by-name contract the outer redex first, the argument is discarded, and the answer is $\lambda y.\, y$. Call-by-value insists on reducing the argument to a value first, and Ω never becomes a value. The same program terminates under one strategy and runs forever under another. You can see the same difference in Rust, where arguments are evaluated even if the function ignores them, and a closure is the standard way to delay evaluation (the call-by-name behavior, written by hand):

```
println!("Rust is call-by-value: both arguments are evaluated first");
let first = |a: &'static str, _b: &'static str| a;
let result = first(arg("a"), arg("b"));
println!("  result: {result}");
println!("A closure delays evaluation, as call-by-name would:");
let first_lazy = |a: &dyn Fn() -> &'static str, _b: &dyn Fn() -> &'static str| a();
let result = first_lazy(&|| arg("a"), &|| arg("b"));
println!("  result: {result}");
```

```
Rust is call-by-value: both arguments are evaluated first
  evaluating the argument "a"
  evaluating the argument "b"
  result: a
A closure delays evaluation, as call-by-name would:
  evaluating the argument "a"
  result: a
```

This is why `Option::unwrap_or_else` takes a closure while `unwrap_or` takes a value: `unwrap_or(expensive())` always runs `expensive()`, call-by-value, while `unwrap_or_else(|| expensive())` runs it only if needed.

The second term duplicates an argument that is itself a redex:

```
(λx. x x) ((λy. y) z)
  normal order   z z   (3 steps)
```

```
  call-by-name   z ((λy. y) z)   (2 steps)
  call-by-value  z z   (2 steps)
```

Normal order substitutes the unevaluated argument, duplicating it, and then has to reduce both copies: three steps. Call-by-value reduces the argument once, before substituting it: two steps. Call-by-name stops early, at $z\,((\lambda y.\, y)\, z)$: the head of the term is the variable z, which is not a function the strategy can call, and call-by-name does not look inside arguments. A term whose head cannot be reduced further like this is in **weak head normal form**; it is where lazy languages stop evaluating, and the remaining redex would be evaluated only if someone needed it. The third term shows the other difference, reduction under λ:

```
λx. (λy. y) x
  normal order   λx. x   (1 step)
  call-by-name   λx. (λy. y) x   (0 steps)
  call-by-value  λx. (λy. y) x   (0 steps)
```

For call-by-value and call-by-name, $\lambda x.\, (\lambda y.\, y)\, x$ is already finished: it is a function, and its body will be evaluated when it is called. That is how every programming language treats function bodies, Rust included. Normal order reduces inside and finds the normal form $\lambda x.\, x$.

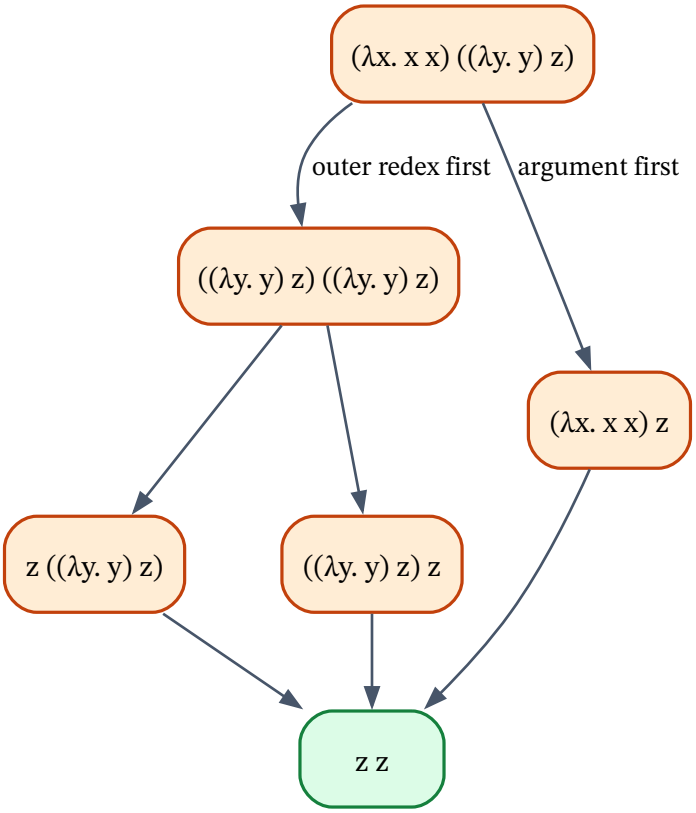
Normal order earns its name with a classical theorem (a consequence of the **standardization theorem**): if a term has a normal form at all, normal order reaches it. Normal order is the safe choice when you want *the* answer of a term. It is not the efficient choice: it can duplicate unevaluated work, as the second example showed. Lazy languages such as Haskell use a refinement, *call-by-need*, that substitutes unevaluated arguments but shares them, so that each one is evaluated at most once.

7.6 Confluence: the answer does not depend on the route

If different strategies can disagree about *whether* a term has an answer, can they disagree about *what* the answer is? No, and this is the most important theorem about the untyped λ-calculus.

> **Theorem (Church–Rosser, 1936).** If $t \xrightarrow{*} u_1$ and $t \xrightarrow{*} u_2$, then there is a term v with $u_1 \xrightarrow{*} v$ and $u_2 \xrightarrow{*} v$.

However two computations diverge, they can always be brought back together. This property is called **confluence**. The diagram shows the complete reduction graph of the second term above: every path ends at $z\,z$.



Every reduction path of (λx. x x) ((λy. y) z) ends in the same normal form z z: the λ-calculus is confluent.

The proof is not hard to understand but long to write down carefully (the standard method, due to Tait and Martin-Löf, goes through "parallel reduction", which contracts any set of redexes at once), and we will not do it. Its most useful consequence is immediate: **a term has at most one normal form**. If $t \xrightarrow{*} n_1$ and $t \xrightarrow{*} n_2$ with both normal, the theorem gives a common v; but a normal form cannot step, so $n_1 = v = n_2$ (up to α). Whatever strategy you use, if it finds an answer, it finds *the* answer. Confluence says nothing about termination, though; Ω is perfectly confluent.

> **NOTE**
>
> Rust has no such theorem, and could not: as soon as a language has side effects, the order of evaluation is observable. `f(print("a"), print("b"))` prints `ab` because Rust specifies left-to-right evaluation of arguments. The pure λ-calculus has no effects, so only termination can tell strategies apart.

7.7 Recursion without names

The λ-calculus has no `fn`, no names for functions, and therefore no obvious way to write a recursive function. A recursive definition such as $fact = \lambda n. \ldots fact\,(n-1) \ldots$ refers to itself, and the calculus has no way to say "itself". Yet recursion is possible, and Ω shows how: Ω manages to reproduce itself by applying a function to a copy of itself. Generalize the trick and you get Curry's **fixed-point combinator**:

$$Y = \lambda f.\,(\lambda x.\,f\,(x\,x))\;(\lambda x.\,f\,(x\,x))$$

Watch what it does to an arbitrary g. Write $W = (\lambda x.\,g\,(x\,x))\;(\lambda x.\,g\,(x\,x))$.

1. $Y\,g \to W$, substituting g for f.
2. $W \to g\,W$, substituting $\lambda x.\,g\,(x\,x)$ for x in $g\,(x\,x)$: the two copies of x become two copies of the same function, and their application is W again.

So $W \to g\,W \to g\,(g\,W) \to \cdots$: W is a **fixed point** of g, a term that g maps to itself (up to reduction). To define factorial, write its body with the recursive call as a parameter,

$$factStep = \lambda fact.\,\lambda n.\,if\;(iszero\;n)\;1\;(times\;n\;(fact\;(pred\;n)))$$

and take its fixed point: $Y\;factStep$ behaves as the factorial, because each time the body needs the recursive call, W unfolds once more. (The helpers if, $iszero$, $times$ and $pred$ are λ-terms too; we build them in the next section.)

Y only works with a lazy strategy. Under call-by-value, step 2 must first evaluate the argument W of g, which requires evaluating W again, forever: the self-application $x\,x$ is evaluated before anyone asks for it. The fix is to hide the self-application under a λ, which call-by-value does not enter. Replacing $x\,x$ by its **η-expansion** $\lambda v.\,x\,x\,v$ (a function that, when called with v, does what $x\,x$ would do) gives the call-by-value fixed-point combinator

$$Z = \lambda f.\,(\lambda x.\,f\,(\lambda v.\,x\,x\,v))\;(\lambda x.\,f\,(\lambda v.\,x\,x\,v))$$

The crate defines both, together with a factorial body for call-by-value whose branches are wrapped in $\lambda d.\,\ldots$ so that only the chosen branch runs (another use of λ to delay evaluation):

```
/// `Ω = (λx. x x) (λx. x x)`: reduces to itself, forever.
pub fn omega() -> Term {
    p("(λx. x x) (λx. x x)")
}

/// Curry's `Y = λf. (λx. f (x x)) (λx. f (x x))`. In two steps `Y g`
/// reduces to `g W`, where `W` is the term `Y g` reached after one step: so
/// `W` and `g W` are equal up to reduction, and `W` is a fixed point of `g`.
pub fn y() -> Term {
    p("λf. (λx. f (x x)) (λx. f (x x))")
}

/// `Z = λf. (λx. f (λv. x x v)) (λx. f (λv. x x v))`: the call-by-value fixed
/// point combinator. The extra `λv` delays the self-application.
pub fn z() -> Term {
    p("λf. (λx. f (λv. x x v)) (λx. f (λv. x x v))")
}
```

```
factorial of 3 under call-by-value:
  with Y: out of fuel after 200 steps; the term grew from 141 to 23536 nodes
  with Z: a value after 175 steps; normalizing it gives the numeral 6
```

With Y, call-by-value never gets anywhere: the term keeps unfolding, and in 200 steps it grew from 141 to more than 23,000 nodes. With Z, it computes a value in 175 steps. Because call-by-value stops at a λ, that value is not yet the numeral 6 but a function that still has work inside it; normal order finishes the job.

In Rust, recursion comes from named `fn` items, which may call themselves. A closure cannot refer to itself directly (it has no name, and, as Chapter 8 shows, its type would have to contain itself), which is why recursive helpers in Rust are written as nested `fn`s or receive themselves as an explicit argument.

7.8 Church encodings

The λ-calculus has no data, only functions. Church's insight was that data can be represented by *what you can do with it*. These representations are called **Church encodings**, and they are the clearest demonstration that three constructs are enough.

Booleans

What do you do with a boolean? You choose between two alternatives. So a boolean *is* a chooser:

$$true = \lambda t.\, \lambda f.\, t \qquad false = \lambda t.\, \lambda f.\, f \qquad if = \lambda b.\, \lambda x.\, \lambda y.\, b\ x\ y$$

$if\ true\ x\ y \xrightarrow{*} true\ x\ y \xrightarrow{*} x$. The if does nothing but pass its arguments to the boolean: a Church boolean is its own conditional. The logical operations follow by choosing well: $and = \lambda a.\, \lambda b.\, a\ b\ false$ ("if a then b, else false"), and similarly for or and not. In the crate:

```
/// `true = λt. λf. t`: of two alternatives, choose the first.
pub fn tru() -> Term {
    p("λt. λf. t")
}

/// `false = λt. λf. f`: of two alternatives, choose the second.
pub fn fls() -> Term {
    p("λt. λf. f")
}

/// `if = λb. λthen. λelse. b then else`: a boolean *is* its own `if`.
pub fn ite() -> Term {
    p("λb. λthen. λelse. b then else")
}

/// `and = λa. λb. a b false`
pub fn and() -> Term {
    with("λa. λb. a b false", &[("false", fls())])
}

/// `or = λa. λb. a true b`
pub fn or() -> Term {
    with("λa. λb. a true b", &[("true", tru())])
}

/// `not = λa. a false true`
pub fn not() -> Term {
    with("λa. a false true", &[("false", fls()), ("true", tru())])
}
```

The helper `with` parses a template and substitutes the given terms for its free variables, so that `and` can mention `false` by name:

```
/// Parses `src` and replaces its free variables by the given terms. The terms
/// are closed, so the substitution never has to rename anything: this is how
/// we write `and` in terms of `false` without a `let` in the calculus.
fn with(src: &str, defs: &[(&str, Term)]) -> Term {
    defs.iter().fold(p(src), |t, (x, def)| t.subst(x, def))
}
```

Pairs

What do you do with a pair? You take it apart. A pair is a function waiting for a selector, and the selectors are the booleans:

$$pair = \lambda a.\, \lambda b.\, \lambda s.\, s\ a\ b \qquad fst = \lambda p.\, p\ true \qquad snd = \lambda p.\, p\ false$$

$fst\ (pair\ a\ b) \xrightarrow{*} pair\ a\ b\ true \xrightarrow{*} true\ a\ b \xrightarrow{*} a$.

Numerals

What do you do with a natural number n? You do something n times. The Church numeral $\overline{n}$ takes a function f and a starting point x and applies f to x, n times:

$$\overline{0} = \lambda f.\, \lambda x.\, x \qquad \overline{1} = \lambda f.\, \lambda x.\, f\ x \qquad \overline{n} = \lambda f.\, \lambda x.\, \underbrace{f\ (f\ (\cdots (f}_{n}\ x)))$$

Arithmetic follows from this one idea:

- $succ = \lambda n.\, \lambda f.\, \lambda x.\, f\ (n\ f\ x)$: do it n times, then once more.
- $plus = \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\ f\ (n\ f\ x)$: do it n times, then m more times.
- $times = \lambda m.\, \lambda n.\, \lambda f.\, m\ (n\ f)$: "do f n times" is a function; do *that* m times.
- $power = \lambda m.\, \lambda n.\, n\ m$: apply m itself n times.
- $iszero = \lambda n.\, n\ (\lambda z.\, false)\ true$: start from $true$; any application of $\lambda z.\, false$ turns it into $false$.

The example `crates/lambda/examples/ch07_church.rs` shows $succ\ \overline{1}$ reduced in normal order, one step per line:

```
succ 1, step by step (normal order):
    (λn. λf. λx. f (n f x)) (λf. λx. f x)
  → λf. λx. f ((λf. λx. f x) f x)
  → λf. λx. f ((λx. f x) x)
  → λf. λx. f (f x)
```

Follow it. The first step substitutes $\overline{1}$ for n in the body of $succ$. Normal order then goes under the two λs, finds that the head f is a variable, and moves into the argument $(\lambda f.\, \lambda x.\, f\ x)\ f\ x$, whose leftmost redex is $(\lambda f.\, \lambda x.\, f\ x)\ f$: the numeral 1 receives the "outer" f (second line). Then $(\lambda x.\, f\ x)\ x$ contracts to $f\ x$, and the result is $\lambda f.\, \lambda x.\, f\ (f\ x) = \overline{2}$.

The predecessor is famously harder: there is no way to "undo" one application of f. The solution, a trick due to Kleene, counts up with pairs: start from $(0, 0)$ and apply $(a, b) \mapsto (b, b + 1)$ n times; the pair becomes $(n - 1, n)$, and its first component is the predecessor. Subtraction is then "apply $pred$ n times":

```rust
/// The predecessor, by Kleene's trick: count up pairs `(k, k + 1)` starting
/// from `(0, 0)`; after n steps the first component is n - 1.
/// `pred = λn. fst (n (λq. pair (snd q) (succ (snd q))) (pair 0 0))`
pub fn pred() -> Term {
    with(
        "λn. fst (n (λq. pair (snd q) (succ (snd q))) (pair zero zero))",
        &[
            ("pair", pair()),
            ("fst", fst()),
            ("snd", snd()),
            ("succ", succ()),
            ("zero", numeral(0)),
        ],
```

```
    )
}

/// `sub = λm. λn. n pred m`: truncated subtraction (`2 - 5 = 0`).
pub fn sub() -> Term {
    with("λm. λn. n pred m", &[("pred", pred())])
}
```

Here are the encodings at work. Each program is reduced to normal form in normal order, and the result is read back as a number or a boolean:

```
program             result   steps
plus 2 3                 5       6
times 2 3                6       7
power 2 3                8      16
pred 3                   2      36
sub 5 2                  3     106
iszero 0              true       3
iszero 2             false       4
and true false       false       4
fst (pair 1 2)           1       6
Y fact_step 3            6    1525
```

The last line is the factorial defined with Y: 1525 β-steps to compute $3! = 6$. The λ-calculus is not an efficient way to multiply, but it is a complete one: every function on natural numbers that a Turing machine can compute can be written as a λ-term on Church numerals. Kleene and Turing showed in the 1930s that the functions definable in the λ-calculus are exactly those that Turing machines compute, one of the pieces of evidence for the Church–Turing thesis.

One more line of the output deserves attention:

```
false = λt. λf. f
0     = λf. λx. x
α-equivalent: true
```

The boolean $false$ and the numeral $\overline{0}$ are the *same term* (up to α). Nothing in the untyped calculus can tell a boolean from a number; $iszero$ applied to $true$ happily produces an answer, and so does $plus$ applied to a pair. The calculus is not unsafe in the sense of crashing (every term is a function, and every function can be applied), but it gives no help at all against meaningless programs. That is the job of types.

Church numerals in Rust

Rust closures can play the same game, with one important difference: they are typed. A Church numeral at the fixed type `u64` is "a function that takes an `f: u64 -> u64` and returns `f` composed with itself n times":

```
/// A function on `u64`, shared so that it can be used many times.
type Fun = Rc<dyn Fn(u64) -> u64>;

/// A Church numeral *at the fixed type `u64`*: given `f`, return `f` composed
/// with itself n times.
type Church = Rc<dyn Fn(Fun) -> Fun>;

/// `0 = λf. λx. x`
fn zero() -> Church {
    Rc::new(|_f: Fun| -> Fun { Rc::new(|x| x) })
```

```
}

/// `succ = λn. λf. λx. f (n f x)`
fn succ(n: Church) -> Church {
    Rc::new(move |f: Fun| -> Fun {
        let n_f = n(f.clone());
        Rc::new(move |x| f(n_f(x)))
    })
}

/// `plus = λm. λn. λf. λx. m f (n f x)`
fn plus(m: Church, n: Church) -> Church {
    Rc::new(move |f: Fun| -> Fun {
        let m_f = m(f.clone());
        let n_f = n(f);
        Rc::new(move |x| m_f(n_f(x)))
    })
}

/// `times = λm. λn. λf. m (n f)`
fn times(m: Church, n: Church) -> Church {
    Rc::new(move |f: Fun| m(n(f)))
}

/// Read a numeral back: apply it to "add one" and start from 0.
fn to_u64(n: &Church) -> u64 {
    n(Rc::new(|x| x + 1))(0)
}
```

```
Church numerals at type u64:
  two = 2, three = 3
  plus(two, three)  = 5
  times(two, three) = 6
```

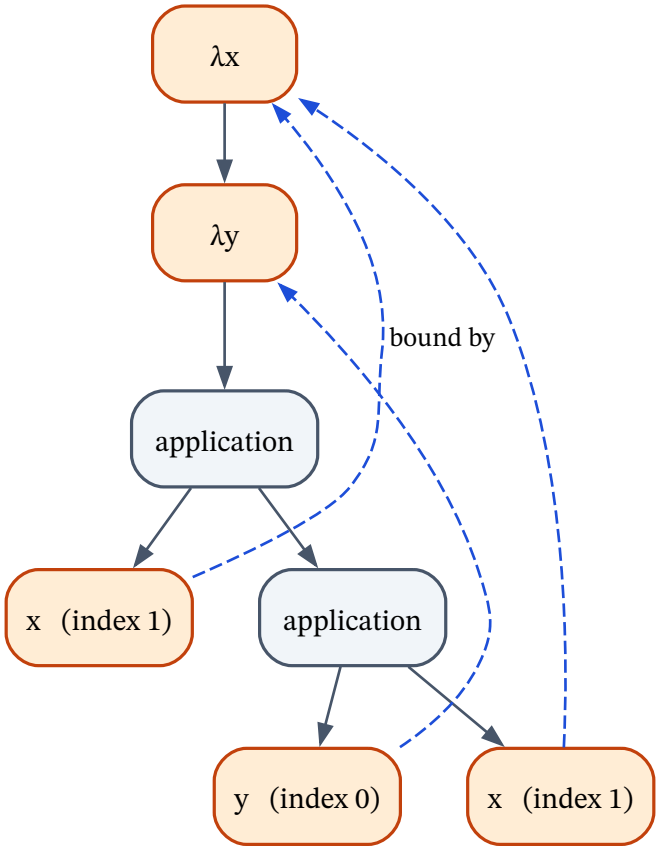
Three things in this code are worth noticing. First, the `Rc<dyn Fn>`: every closure in Rust has its own type (Chapter 8), so to give "all numerals" a single type we put them behind a pointer to a trait object. Second, the `f.clone()` in `succ` and `plus`: the λ-term $\lambda f.\, \lambda x.\, f\,(n\,f\,x)$ uses f twice, and Rust makes that duplication explicit, as it makes every duplication explicit. Third, what is missing: `power`. In the untyped calculus $power = \lambda m.\, \lambda n.\, n\,m$ applies the numeral n to the numeral m, which means using n at the type "functions on numerals" rather than "functions on `u64`". A numeral that works at *every* type at once needs parametric polymorphism of a kind Rust does not have for closures; Chapter 12 explains what is missing and how far traits can simulate it.

7.9 De Bruijn indices: names are the problem

Capture-avoiding substitution works, but it is fiddly: we must compute free variables, invent fresh names, and remember that the result is only defined up to α. Every implementation of a language with binders faces this, and the Dutch mathematician N. G. de Bruijn proposed a radical solution in the early 1970s, while working on Automath, one of the first proof checkers: get rid of the names.

A bound variable is replaced by a number, its **de Bruijn index**: how many λs you must cross, going outward from the occurrence, to reach its binder. The binders themselves carry no name at all:

$$\lambda x.\, x = \lambda.\, 0 \qquad \lambda x.\, \lambda y.\, x = \lambda.\, \lambda.\, 1 \qquad \lambda x.\, \lambda y.\, x\,(y\,x) = \lambda.\, \lambda.\, 1\,(0\,1)$$



The syntax tree of λx. λy. x (y x). Dashed arrows connect each variable to the λ that binds it; the de Bruijn index counts the λs crossed on the way up.

The same variable can have different indices at different places (in $\lambda x.\, x\,(\lambda y.\, x)$ the two occurrences of x are 0 and 1), and different variables can share an index. What the indices capture is the *binding structure* and nothing else. Free variables are numbered beyond the binders, according to an agreed list of free names (the **naming context**): in the context $[z]$, the term $\lambda x.\, x\, z$ is $\lambda.\, 0\, 1$.

The payoff is immediate: **α-equivalent terms have identical de Bruijn representations**, because renaming bound variables does not change any index. The crate's `alpha_eq` simply converts both terms and compares them with `==`:

```
α-equivalence is equality of de Bruijn terms:
  λx. λy. x  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
  λa. λb. a  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
  λx. λy. y  ↦ λ. λ. 0    α-equivalent to λx. λy. x: false
  λy. λx. y  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
```

The conversion walks the term with a stack of the enclosing binders' names; a variable's index is its distance from the top of the stack, which also handles shadowing correctly (the nearest binder wins):

```rust
/// Converts a named term to de Bruijn form. `free` lists the names of the free
/// variables: the free variable `free[i]` becomes index `i` (plus the number of
/// enclosing binders).
pub fn to_de_bruijn(t: &Term, free: &[&str]) -> Result<DbTerm, String> {
    // `scope` holds the names of the enclosing binders, innermost last.
    fn go(t: &Term, scope: &mut Vec<String>, free: &[&str]) -> Result<DbTerm, String> {
```

```
        match t {
            Term::Var(x) => {
                // The nearest binder named x wins: that is how shadowing works.
                if let Some(pos) = scope.iter().rev().position(|y| y == x) {
                    Ok(DbTerm::Var(pos))
                } else if let Some(i) = free.iter().position(|y| y == x) {
                    Ok(DbTerm::Var(scope.len() + i))
                } else {
                    Err(format!("free variable `{x}` is not in the naming context"))
                }
            }
            Term::Lam(x, body) => {
                scope.push(x.clone());
                let body = go(body, scope, free);
                scope.pop();
                Ok(db_lam(body?))
            }
            Term::App(f, a) => Ok(db_app(go(f, scope, free)?, go(a, scope, free)?)),
        }
    }
    go(t, &mut Vec::new(), free)
}
```

Shifting and substitution

Names are gone, but substitution still needs care, because indices are *relative*: the same variable has a different index depending on how many binders surround it. Two operations handle this.

Shifting. When a term is moved under an extra binder, its free variables must point one level further out. The function shift_c adds one to every index that is at least the **cutoff** c; indices below the cutoff are bound inside the term and stay put:

- $\mathit{shift}_c(k) = k$ if $k < c$, and $k + 1$ if $k \geq c$;
- $\mathit{shift}_c(\lambda.\, t) = \lambda.\, \mathit{shift}_{c+1}(t)$ (one more binder, so the cutoff grows);
- $\mathit{shift}_c(t\ u) = \mathit{shift}_c(t)\ \mathit{shift}_c(u)$.

Substitution. $t[j := s]$ replaces index j by s. For β-reduction we also want the indices above j to *decrease* by one, because the λ that bound j disappears:

- $k[j := s] = k$ if $k < j$, s if $k = j$, and $k - 1$ if $k > j$;
- $(\lambda.\, t)[j := s] = \lambda.\, t[j + 1 := \mathit{shift}_0(s)]$;
- $(t\ u)[j := s] = t[j := s]\ u[j := s]$.

Under a binder, the variable we are looking for is one level further away (so $j + 1$), and the replacement s must be shifted, so that its free variables skip the binder we just entered. That shift is precisely what capture-avoiding renaming did with names. β-reduction becomes

$$(\lambda.\, t)\ s \to t[0 := s]$$

and there are no names to clash, no fresh names to invent. The Rust code is a line-by-line transcription:

```
/// Adds one to every index that is at least `cutoff`. Indices below the
/// cutoff are bound inside the term and do not move; the others are free.
pub fn shift(&self, cutoff: usize) -> DbTerm {
    match self {
        DbTerm::Var(k) if *k < cutoff => DbTerm::Var(*k),
        DbTerm::Var(k) => DbTerm::Var(k + 1),
        DbTerm::Lam(body) => db_lam(body.shift(cutoff + 1)),
        DbTerm::App(t, u) => db_app(t.shift(cutoff), u.shift(cutoff)),
```

```
    }
}
```

```
/// `self[j := s]`: replaces index `j` by `s` and lowers the free indices
/// above `j` by one (the binder of `j` is about to disappear). Each time we
/// go under a `λ`, `j` grows by one and `s` is shifted, so that the free
/// variables of `s` keep pointing past the new binder.
pub fn subst(&self, j: usize, s: &DbTerm) -> DbTerm {
    match self {
        DbTerm::Var(k) if *k < j => DbTerm::Var(*k),
        DbTerm::Var(k) if *k == j => s.clone(),
        DbTerm::Var(k) => DbTerm::Var(k - 1),
        DbTerm::Lam(body) => db_lam(body.subst(j + 1, &s.shift(0))),
        DbTerm::App(t, u) => db_app(t.subst(j, s), u.subst(j, s)),
    }
}
```

> **NOTE**
>
> Pierce's *Types and Programming Languages* defines a general shift shift_c^d by any amount d, possibly negative, a substitution that does not lower indices, and β as "shift up the argument, substitute, shift the result down". Our version folds the lowering into the substitution, so only upward shifts are ever needed, which suits natural numbers (`usize` in Rust, `Nat` in Lean). The two definitions give the same results.

Now the capture example once more, nameless. In the naming context $[y]$, the term $(\lambda x.\, \lambda y.\, x)\, y$ is $(\lambda.\, \lambda.\, 1)\, 0$. The β-step substitutes 0 for index 0 in the body $\lambda.\, 1$:

1. The body is an abstraction, so we go under it: we now look for index 1, and the replacement 0 is shifted to $\mathit{shift}_0(0) = 1$.
2. Inside, we find index 1, which is the one we are looking for: replace it by the shifted replacement, 1.
3. The result is $\lambda.\, 1$: under one binder, index 1 points past it, to the free variable y. It is $\lambda y'.\, y$, exactly what capture-avoiding substitution produced.

If you forget the shift in step 1, the replacement stays 0, and the result is $\lambda.\, 0$, the identity: the capture bug, back again in a new notation.

```
(λx. λy. x) y  is  (λ. λ. 1) 0  in the naming context [y]
  with the shift:    λ. 1
  without the shift: λ. 0
```

De Bruijn indices turn the capture problem into arithmetic that is easy to get right once and for all, which is why most proof assistants, Lean included, use them internally. Their cost is readability, so real implementations keep names for printing. The crate checks that both representations compute the same thing: the de Bruijn reducer takes the same 1525 steps on the factorial and reaches the same normal form.

```
Y fact_step 3 with de Bruijn indices: 1525 steps, same normal form: true
```

7.10 The λ-calculus in Lean

Now we formalize the calculus in Lean, in `code/lean/Book/Ch07/Untyped.lean` and `code/lean/Book/Ch07/Normalize.lean` (the second imports the first). We use de Bruijn indices from the start; you have just seen why.

Terms, shifting and substitution

A term is an inductive type with three constructors, as in Rust. Lean needs no `Box`: an inductive type may refer to itself directly, and Lean's runtime represents the values as pointers anyway.

```
/-- Terms of the untyped λ-calculus with de Bruijn indices: `var k` refers to
the binder `k` levels up. -/
inductive Term where
  | var : Nat → Term
  | lam : Term → Term
  | app : Term → Term → Term
  deriving DecidableEq
```

The file also defines a printer that writes terms the way the Rust crate does, and a few named terms. (`deriving DecidableEq` asks Lean to generate a decision procedure for equality of terms, which we will use later.)

```
/-- `λx. x` -/
def I : Term := lam (var 0)
/-- `λx. λy. x` -/
def K : Term := lam (lam (var 1))
/-- `λx. x x` -/
def delta : Term := lam (app (var 0) (var 0))
/-- `Ω = (λx. x x) (λx. x x)` -/
def omega : Term := app delta delta

/-- info: λ. λ. 1 -/
#guard_msgs in
#eval K

/-- info: (λ. 0 0) (λ. 0 0) -/
#guard_msgs in
#eval omega
```

`shift` and `subst` are the definitions from the previous section, written by pattern matching. Each recursive call is on a direct subterm, so Lean accepts them as **structural recursion** without further ado (Chapter 2 explains how Lean checks this):

```
/-- `shift c t` adds one to every index of `t` that is at least `c`. The
indices below the cutoff `c` are bound inside `t`; the others are free. -/
def shift (c : Nat) : Term → Term
  | var k => if k < c then var k else var (k + 1)
  | lam t => lam (shift (c + 1) t)
  | app t u => app (shift c t) (shift c u)
```

```
/-- `subst j s t` replaces the index `j` in `t` by `s`, and lowers the free
indices above `j` by one, because the binder of `j` disappears. Under a
binder, `j` grows by one and `s` is shifted. -/
def subst (j : Nat) (s : Term) : Term → Term
  | var k => if k < j then var k else if k = j then s else var (k - 1)
  | lam t => lam (subst (j + 1) (shift 0 s) t)
  | app t u => app (subst j s t) (subst j s u)
```

Two facts about them, proved by `rfl` because both sides compute to the same term: the nameless capture example, and the step that turns Ω into itself.

```
/-- `(λx. λy. x) z`, with `z` free (index 0): the body `λ. 1` becomes `λ. 1`
again, the constant function returning `z`. -/
example : subst 0 (var 0) (lam (var 1)) = lam (var 1) := rfl

/-- Contracting `Ω`'s redex rebuilds `Ω`. -/
example : subst 0 delta (app (var 0) (var 0)) = omega := rfl
```

Reduction as an inductive relation

The four rules Beta, AppL, AppR and Abs become the four constructors of an inductive predicate, exactly as typing rules did in Chapter 3. A value of type `Step t u` is a derivation that t reduces to u in one step:

```
/-- One step of β-reduction, anywhere in a term. -/
inductive Step : Term → Term → Prop where
  /-- `(λ. t) s` steps to `t[0 := s]`. -/
  | beta (t s : Term) : Step (app (lam t) s) (subst 0 s t)
  /-- Reduce inside the function part of an application. -/
  | appL {t t' : Term} (s : Term) : Step t t' → Step (app t s) (app t' s)
  /-- Reduce inside the argument of an application. -/
  | appR (t : Term) {s s' : Term} : Step s s' → Step (app t s) (app t s')
  /-- Reduce inside the body of a λ. -/
  | body {t t' : Term} : Step t t' → Step (lam t) (lam t')
```

For "many steps" we use a definition from Mathlib. `Relation.ReflTransGen r` is the **reflexive-transitive closure** of a relation `r`: `ReflTransGen r a b` holds when you can get from `a` to `b` in zero or more `r`-steps. It is defined as an inductive predicate with two constructors, `refl` (zero steps) and `tail` (some steps, then one more), and Mathlib proves the expected lemmas about it, among them `single` (one step is many steps), `head` (one step followed by many) and `trans`. We import it with `import Mathlib.Logic.Relation`.

```
/-- Many steps: the reflexive-transitive closure of `Step`, from Mathlib. -/
abbrev Steps : Term → Term → Prop := Relation.ReflTransGen Step

/-- A term is in normal form when it cannot take any step. -/
def Normal (t : Term) : Prop := ∀ t', ¬ Step t t'
```

A first proof: $K\,a\,b \xrightarrow{*} a$, with a and b the free variables 0 and 1.

```
/-- `K a b` reduces to `a` in two steps (here `a` and `b` are the free
variables 0 and 1). -/
example : Steps (app (app K (var 0)) (var 1)) (var 0) := by
  -- First step: contract `K (var 0)` inside the left part.
  apply Relation.ReflTransGen.head (Step.appL _ (Step.beta _ _))
  -- Second step: contract the remaining redex `(λ. 1) (var 1)`.
  apply Relation.ReflTransGen.single
  exact Step.beta _ _
```

The goal is a `Steps` statement, and we build the sequence from the front. `Relation.ReflTransGen.head` takes a first step and leaves the rest as a new goal; we give it the derivation `Step.appL _ (Step.beta _ _)`, which says "contract the redex $K\,(\texttt{var}\,0)$ inside the left part". The underscores are filled in by unification: Lean works out the terms from the goal. Notice that we never wrote down the intermediate term: Lean computed `subst 0 (var 0) (lam (var 1))` itself, and it is

`lam (var 1)`, the constant function returning variable 0 (which is index 1 under the binder). What remains is `Steps (app (lam (var 1)) (var 1)) (var 0)`, and `single` turns it into one more `Step`, a Beta step whose substitution drops the argument and lowers index 1 to 0.

Ω, formally

Ω steps to itself:

```
/-- `Ω` steps to itself. -/
theorem omega_step : Step omega omega :=
  Step.beta (app (var 0) (var 0)) delta
```

The proof is a Beta step whose stated result is `subst 0 delta (app (var 0) (var 0))`; Lean accepts it as a proof of `Step omega omega` because that substitution *computes* to `omega` (the two are **definitionally equal**). More interesting is that Ω can *only* step to itself, so it never reaches a normal form. These are statements about *all* possible reductions, the kind of fact a test cannot establish:

```
/-- Moreover, the only step `Ω` can take leads back to `Ω`. -/
theorem omega_step_inv {t : Term} (h : Step omega t) : t = omega := by
  cases h with
  | beta => rfl
  | appL _ h =>
    -- A step inside `delta = λ. 0 0`: impossible, `0 0` has no redex.
    cases h with
    | body h =>
      cases h with
      | appL _ h => cases h
      | appR _ h => cases h
  | appR _ h =>
    cases h with
    | body h =>
      cases h with
      | appL _ h => cases h
      | appR _ h => cases h

/-- So every reduction sequence from `Ω` stays at `Ω`. -/
theorem omega_steps_inv {t : Term} (h : Steps omega t) : t = omega := by
  induction h with
  | refl => rfl
  | tail _ hstep ih =>
    subst ih
    exact omega_step_inv hstep

/-- `Ω` has no normal form. -/
theorem omega_no_normal_form : ¬ ∃ t, Steps omega t ∧ Normal t := by
  rintro ⟨t, hsteps, hnormal⟩
  rw [omega_steps_inv hsteps] at hnormal
  exact hnormal omega omega_step
```

The first theorem is proved by **case analysis on the derivation** `h`: `cases h` asks which rule could have produced `Step omega t`. Rule `beta` fits, with `t` forced to be the substitution, which computes to `omega` (`rfl`). Rule `appL` fits only if `delta` itself steps; a second `cases` finds that only `body` applies, then that `app (var 0) (var 0)` would have to step, and finally that no rule lets a variable step: the case is impossible and disappears. Rule `appR` is the same, for the argument. `omega_steps_inv` extends this to many steps by induction on the `ReflTransGen` derivation, and `omega_no_normal_form` combines the two: a reachable normal term would be `omega` (the `rw`), and `omega` is not normal because it steps.

A normalizer, and a proof that it is right

Next, a function that performs one step of normal-order reduction, the Lean twin of the crate's `step_normal`:

```
/-- One step of normal-order reduction: contract the leftmost-outermost redex,
or return `none` if there is no redex. -/
def step : Term → Option Term
  | var _ => none
  | lam t => (step t).map lam
  | app (lam t) s => some (subst 0 s t)
  | app t s =>
    match step t with
    | some t' => some (app t' s)
    | none => (step s).map (app t)
```

The patterns overlap: `app (lam t) s` is listed before the general `app t s`, and Lean, like Rust's `match`, tries them in order. We now prove that this function agrees with the relation `Step`: every step it takes is a `Step` (soundness), and it finds a step whenever one exists (completeness). Soundness first:

```
/-- Every step that `step` takes is a `Step`. -/
theorem step_sound {t u : Term} (h : step t = some u) : Step t u := by
  fun_induction step t generalizing u with
  | case1 =>
    -- `step (var k) = none`, so `h : none = some u` is impossible.
    contradiction
  | case2 t ih =>
    -- `h : Option.map lam (step t) = some u`: the body stepped to some `t'`.
    obtain ⟨t', ht', rfl⟩ := Option.map_eq_some_iff.mp h
    exact Step.body (ih ht')
  | case3 t s =>
    -- A redex: `h : some (subst 0 s t) = some u`.
    cases h
    exact Step.beta t s
  | case4 t s _ t' ht ih =>
    -- The function part stepped: `ht : step t = some t'`.
    cases h
    exact Step.appL s (ih ht)
  | case5 t s _ _ _ ih =>
    -- The function part is normal and the argument stepped.
    obtain ⟨s', hs', rfl⟩ := Option.map_eq_some_iff.mp h
    exact Step.appR t (ih hs')
```

The tactic `fun_induction step t` is **functional induction**: an induction that follows the branches of the *function* `step` rather than the constructors of the *type* `Term`. When we defined `step`, Lean generated an induction principle with one case per branch (`case1` to `case5`, in order) and an induction hypothesis per recursive call, so each case knows which branch `step` took. In `case1` the hypothesis `none = some u` is absurd (`contradiction`). In `case2`, the core lemma `Option.map_eq_some_iff` turns `Option.map lam (step t) = some u` into `step t = some t'` with `u = lam t'`; `obtain` unpacks it (the `rfl` pattern replaces `u` by `lam t'`), and rule `body` lifts the induction hypothesis under the λ. In `case3`, `cases h` on `some (subst 0 s t) = some u` identifies `u` with the substitution, and `beta` applies. `case4` and `case5` use `appL` and `appR` the same way.

Completeness is proved by induction on the derivation of `Step t u`, showing in each case that `step t` is `some _`. The only subtlety is the overlapping pattern: to unfold `step` on an application, Lean must know that its function part is not a λ, and the lemma `step_app` packages that fact (`step.eq_4` is the equation Lean generated for the fourth branch of `step`):

```
/-- How `step` unfolds on an application whose function part is not a `λ`. -/
theorem step_app {t s : Term} (ht : ∀ b, t ≠ lam b) :
    step (app t s) =
      match step t with
      | some t' => some (app t' s)
      | none => (step s).map (app t) := by
  rw [step.eq_4]
  exact fun b h => ht b h
```

```
/-- If any `Step` is possible, `step` finds one: `step` returns `none` only on
normal forms. -/
theorem step_complete {t u : Term} (h : Step t u) : (step t).isSome := by
  induction h with
  | beta t s =>
    -- A redex at the root: the third equation of `step` applies.
    simp [step]
  | @appL t t' s _ ih =>
    -- `t` can step, so `step t` is `some _` (the induction hypothesis).
    cases t with
    | var k => simp [step] at ih
    | lam b => simp [step]
    | app t₁ t₂ =>
      obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
      rw [step_app (by simp), hu]
      rfl
  | @appR t s s' _ ih =>
    -- `s` can step. Either `t` is a `λ` (then the root is a redex), or
    -- `step` tries `t` first and, if `t` is normal, steps `s`.
    cases t with
    | lam b => simp [step]
    | _ =>
      rw [step_app (by simp)]
      split
      · rfl
      · obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
        simp [hu]
  | body _ ih =>
    obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
    simp [step, hu]

/-- So `step t = none` means that `t` is in normal form. -/
theorem normal_of_step_none {t : Term} (h : step t = none) : Normal t := by
  intro t' hstep
  have := step_complete hstep
  rw [h] at this
  contradiction
```

In the `appL` and `appR` cases, `cases t` separates a λ in function position (the root is then a redex, and `simp [step]` sees that `step` returns `some`) from the other shapes, where `step_app` applies; its side condition, "this is not a λ", is discharged by `simp`. The corollary `normal_of_step_none` is what we need: if `step` returns `none`, the term is normal.

With these, a **fuel-based normalizer** is easy to write and to prove correct. It takes a number of steps as an extra argument and decreases it at each step, so Lean sees structural recursion on the fuel:

```
/-- Normal-order reduction for at most `fuel` steps: `some u` if a normal form
`u` was reached, `none` if the fuel ran out. -/
def normalize : Nat → Term → Option Term
  | 0, _ => none
  | fuel + 1, t =>
    match step t with
    | none => some t
    | some t' => normalize fuel t'
```

```
/-- What `normalize` returns is a normal form of its input. -/
theorem normalize_sound {fuel : Nat} {t u : Term} (h : normalize fuel t = some u) :
    Steps t u ∧ Normal u := by
  induction fuel generalizing t with
  | zero =>
    -- No fuel: `normalize 0 t = none`, so `h` is impossible.
    simp [normalize] at h
  | succ fuel ih =>
    cases hs : step t with
    | none =>
      -- `t` is already normal and `normalize` returned it.
      simp [normalize, hs] at h
      subst h
      exact ⟨Relation.ReflTransGen.refl, normal_of_step_none hs⟩
    | some t' =>
      -- One step to `t'`, then the induction hypothesis for the rest.
      simp [normalize, hs] at h
      obtain ⟨hsteps, hnormal⟩ := ih h
      exact ⟨Relation.ReflTransGen.head (step_sound hs) hsteps, hnormal⟩
```

The proof is an induction on the fuel. With zero fuel the hypothesis is false (`simp [normalize] at h` reduces it to `False`). With `fuel + 1`, `cases hs : step t` splits on what `step t` returned and remembers it as `hs`. If `none`, `normalize` returned `t`, which reaches itself in zero steps and is normal by `normal_of_step_none`. If `some t'`, the induction hypothesis covers the rest of the run from `t'`, and `head` prepends the first step, justified by `step_sound`.

Computing proofs

Now Church numerals, in de Bruijn form (the comments give the named version):

```
/-- `f (f (... (f x)))` with `n` copies of `f`, where `f` is the variable `k`
and `x` is the variable `0`. -/
def iterApp (k : Nat) : Nat → Term
  | 0 => var 0
  | n + 1 => app (var k) (iterApp k n)

/-- The Church numeral `n = λf. λx. f (... (f x))`: `f` is index 1, `x` is 0. -/
def church (n : Nat) : Term := lam (lam (iterApp 1 n))

/-- `succ = λn. λf. λx. f (n f x)` -/
def succ : Term := lam (lam (lam (app (var 1) (app (app (var 2) (var 1)) (var 0)))))

/-- `plus = λm. λn. λf. λx. m f (n f x)` -/
def plus : Term :=
  lam (lam (lam (lam (app (app (var 3) (var 1)) (app (app (var 2) (var 1)) (var 0))))))

/-- `times = λm. λn. λf. m (n f)` -/
def times : Term := lam (lam (lam (app (var 2) (app (var 1) (var 0)))))
```

```
/-- info: λ. λ. 1 (1 (1 0)) -/
#guard_msgs in
#eval church 3

/-- info: some (λ. λ. 1 (1 (1 (1 (1 0))))) -/
#guard_msgs in
#eval normalize 100 (app (app plus (church 2)) (church 3))

/-- info: none -/
#guard_msgs in
#eval normalize 100 omega
```

`#eval` runs `normalize`: $2 + 3$ normalizes to the numeral 5, and Ω exhausts its fuel. The soundness theorem turns such computations into proofs:

```
/-- `2 + 3` reduces to `5`: `normalize` computes the answer, and
`normalize_sound` turns the computation into a proof. -/
theorem two_plus_three : Steps (app (app plus (church 2)) (church 3)) (church 5) :=
  (normalize_sound (fuel := 100) (by decide)).1

theorem two_times_three : Steps (app (app times (church 2)) (church 3)) (church 6) :=
  (normalize_sound (fuel := 100) (by decide)).1
```

`normalize_sound` needs a proof that `normalize 100 (...) = some (church 5)`. Since `Term` has decidable equality, that equation is a decidable proposition, and `decide` asks Lean's kernel (the small trusted core that checks every proof) to evaluate it. This pattern, "write a procedure, prove it sound, run it", is called **proof by reflection**, and many large automated proofs work this way.

For specific numbers, computation is enough. For *all* numbers, we need a real proof. Here is one: $succ\ \overline{n} \xrightarrow{*} \overline{n+1}$ for every n.

```
/-- Church numerals are closed, so shifting does not change them. -/
theorem shift_iterApp (c n : Nat) : shift (c + 2) (iterApp 1 n) = iterApp 1 n := by
  induction n with
  | zero => simp [iterApp, shift]
  | succ n ih => simp [iterApp, shift, ih]

theorem shift_church (c n : Nat) : shift c (church n) = church n := by
  simp [church, shift, shift_iterApp]

/-- Substituting for `f` (index 1) renames it to index 2... -/
theorem subst_iterApp_f (n : Nat) :
    subst 1 (var 2) (iterApp 1 n) = iterApp 2 n := by
  induction n with
  | zero => simp [iterApp, subst]
  | succ n ih => simp [iterApp, subst, ih]

/-- ...and substituting `x` (index 0) for itself lowers index 2 back to 1. -/
theorem subst_iterApp_x (n : Nat) :
    subst 0 (var 0) (iterApp 2 n) = iterApp 1 n := by
  induction n with
  | zero => simp [iterApp, subst]
  | succ n ih => simp [iterApp, subst, ih]
```

```
/-- Reduction sequences can be carried out under a `λ`... -/
theorem steps_body {t t' : Term} (h : Steps t t') : Steps (lam t) (lam t') := by
  induction h with
  | refl => exact Relation.ReflTransGen.refl
  | tail _ hstep ih => exact ih.tail (Step.body hstep)

/-- ...and inside the argument of an application. -/
theorem steps_appR (t : Term) {s s' : Term} (h : Steps s s') :
    Steps (app t s) (app t s') := by
  induction h with
  | refl => exact Relation.ReflTransGen.refl
  | tail _ hstep ih => exact ih.tail (Step.appR t hstep)
```

```
/-- For every `n`, `succ (church n)` reduces to `church (n + 1)`. -/
theorem succ_correct (n : Nat) : Steps (app succ (church n)) (church (n + 1)) := by
  -- Step 1, the outer redex: `succ (church n)` becomes `λ. λ. 1 (church n 1 0)`.
  have h1 : Step (app succ (church n))
      (lam (lam (app (var 1) (app (app (church n) (var 1)) (var 0))))) := by
    have := Step.beta (lam (lam (app (var 1) (app (app (var 2) (var 1)) (var 0)))))
      (church n)
    simp [subst, shift_church] at this
    exact this
  -- Step 2, inside: `church n 1` becomes `λ. iterApp 2 n`.
  have h2 : Step (app (app (church n) (var 1)) (var 0))
```

```
      (app (lam (iterApp 2 n)) (var 0)) := by
    have := Step.appL (var 0) (Step.beta (lam (iterApp 1 n)) (var 1))
    simp [subst, shift, subst_iterApp_f] at this
    exact this
  -- Step 3: `(λ. iterApp 2 n) 0` becomes `iterApp 1 n`.
  have h3 : Step (app (lam (iterApp 2 n)) (var 0)) (iterApp 1 n) := by
    have := Step.beta (iterApp 2 n) (var 0)
    rwa [subst_iterApp_x] at this
  -- Steps 2 and 3 happen under two binders, in the argument of `1`.
  have h23 : Steps (app (app (church n) (var 1)) (var 0)) (iterApp 1 n) :=
    Relation.ReflTransGen.head h2 (Relation.ReflTransGen.single h3)
  -- `church (n + 1)` is, by definition, `λ. λ. 1 (iterApp 1 n)`.
  exact Relation.ReflTransGen.head h1 (steps_body (steps_body (steps_appR _ h23)))
```

The proof follows the reduction you saw in the Rust trace, three steps, but for a symbolic n:

- **Step 1** contracts the outer redex. We take the Beta step as `this` and let `simp` compute its substitution; since `church n` is closed, shifting it changes nothing (`shift_church`, by induction on `n`). `exact this` matches the result against `h1`.
- **Step 2** contracts `church n 1`. The substitution turns the numeral's own `f` (index 1) into index 2, the outer f seen from under the numeral's remaining binder (`subst_iterApp_f`, again by induction).
- **Step 3** substitutes x for x, lowering index 2 back to 1 (`subst_iterApp_x`).

Steps 2 and 3 happen under two λs and in the argument of f; `steps_body` and `steps_appR` carry a reduction sequence into such a context, each by a short induction. The final term is `lam (lam (app (var 1) (iterApp 1 n)))`, which is `church (n + 1)` by definition, so Lean accepts it as it is.

Why Lean insists on fuel

Why not write a normalizer without fuel, one that simply keeps stepping until it reaches a normal form? Here is the attempt, and Lean's answer:

```
/--
error: fail to show termination for
  Ch07.normalizeAll
with errors
failed to infer structural recursion:
Cannot use parameter t:
  failed to eliminate recursive application
    normalizeAll t'


failed to prove termination, possible solutions:
  - Use `have`-expressions to prove the remaining goals
  - Use `termination_by` to specify a different well-founded relation
  - Use `decreasing_by` to specify your own tactic for discharging this kind of goal
t t' : Term
⊢ sizeOf t' < sizeOf t
-/
#guard_msgs in
def normalizeAll (t : Term) : Term :=
  match step t with
  | none => t
  | some t' => normalizeAll t'
```

Lean requires every function to terminate, and it must see why. It first tried **structural recursion**, but `t'` is the result of a reduction step, not a subterm of `t`. It then tried **well-founded recursion**

on the size of the term (`sizeOf`) and left the goal it could not prove: `sizeOf t' < sizeOf t`. That goal is false, since a β-step can make a term bigger (you saw Y grow from 141 to 23,536 nodes). In fact *no* measure can work, because the function does not terminate on Ω. Lean is not being fussy; it is right. And it must be strict: if it accepted `def loop (n : Nat) : Nat := loop n + 1`, the equation `loop 0 = loop 0 + 1` would prove `0 = 1` (Chapter 10 makes this precise). What Lean offers instead is `partial`:

```
/-- The same function, marked `partial`: Lean accepts it without a termination
proof, compiles it, and lets us run it. -/
partial def normalizeAll' (t : Term) : Term :=
  match step t with
  | none => t
  | some t' => normalizeAll' t'

/-- info: λ. λ. 1 (1 (1 (1 (1 0)))) -/
#guard_msgs in
#eval normalizeAll' (app (app times (church 2)) (church 3))
```

A `partial def` is compiled and runs, so `#eval` works (and would loop forever on Ω). The price is that the logic knows nothing about it. Lean only requires its type to be inhabited (here it is: `fun t => t` has type `Term → Term`), and then treats the function as an *opaque* constant: you cannot unfold it in a proof, even on an input where it obviously returns at once.

```
/--
error: Tactic `rfl` failed: The left-hand side
  normalizeAll' (var 0)
is not definitionally equal to the right-hand side
  var 0

⊢ normalizeAll' (var 0) = var 0
-/
#guard_msgs in
example : normalizeAll' (var 0) = var 0 := by
  rfl
```

So the choice in Lean is: fuel, which makes the function total and lets you prove theorems about it (`normalize_sound`), or `partial`, which makes it runnable and nothing more. Rust makes the second choice for every function: `loop` and unrestricted recursion are always available, and termination is never checked. Chapter 8 shows that adding types to the λ-calculus brings termination back, for free.

7.11 Rust closures are typed λ-terms

Rust's closures are λ-abstractions, and calling a closure is β-reduction in call-by-value order. Here are the combinators from the beginning of the chapter, written in Rust:

```
/// `λx. x`
fn identity<A>(x: A) -> A {
    x
}

/// `λx. λy. x`: given x, return a closure that ignores its argument.
fn konst<A: Clone, B>(x: A) -> impl Fn(B) -> A {
    move |_y| x.clone()
}

/// `λf. λg. λx. g (f x)`: first f, then g.
```

```
fn compose<A, B, C>(f: impl Fn(A) -> B, g: impl Fn(B) -> C) -> impl Fn(A) -> C {
    move |x| g(f(x))
}
```

```
identity(7)              = 7
konst("kept")(42)        = kept
compose(inc, double)(5)  = 12
```

The difference is in the signatures: every parameter and every result has a type. Most of the time the types are generic, so the combinators work for any types `A`, `B` and `C`, but each use of a combinator picks specific ones. That restriction is what rules out the strange terms of this chapter. Try to write the self-application $\delta = \lambda x.\, x\, x$ as a closure:

```
pub fn delta() {
    let self_apply = |x| x(x);
}
```

```
error[E0282]: type annotations needed
 --> compile_fail/ch07_self_application.rs:7:23
  |
7 |     let self_apply = |x| x(x);
  |                       ^  - type must be known at this point
  |
help: consider giving this closure parameter an explicit type
  |
7 |     let self_apply = |x: /* Type */| x(x);
  |                        ++++++++++++
```

The compiler must know the type of `x` to call it, and nothing tells it. But annotations cannot help either. Suppose `x` had some type T. Since `x` is called, T must be a function type, say $A \to B$. Since `x` is its own argument, A must be T. So $T = T \to B$: a type that contains itself as a proper part, which no finite type does. Chapter 8 turns this argument into a theorem (proved in Lean), and Chapter 13 shows the same failure in type inference, where it is detected by a check called the *occurs check*. Without δ there is no Ω, no Y and no Z, and, as Chapter 8 shows, no non-termination at all.

7.12 Putting it together: the factorial of 3

Let us retrace how the crate computes 3!, the last line of the arithmetic table, because it uses nearly every idea of this chapter.

1. **Encoding.** The numerals, $iszero$, $times$ and $pred$ are Church encodings; $pred$ uses Church pairs, and the conditional is a Church boolean. The `with` helper assembles them by substituting closed terms for names, so no renaming is ever needed there.
2. **Recursion.** `fact_step` abstracts the body of factorial over the recursive call, and Y ties the knot: $Y\ factStep$ unfolds once for every recursive call.
3. **Strategy.** Normal order finds the answer in 1525 steps. Call-by-value loops with Y; with Z and delayed branches it computes a value, which normal order then tidies up into the numeral.
4. **Substitution and confluence.** Every step is a capture-avoiding substitution, and by the Church–Rosser theorem any strategy that reaches a normal form reaches this one, $\overline{6}$.

5. **Two representations.** The de Bruijn reducer, which never renames anything, makes the same choices and reaches the same normal form.
6. **Reading back.** `decode_numeral` recognizes $\lambda f.\, \lambda x.\, f\,(\cdots(f\,x))$ and counts the fs. It must be told to expect a number, because $false$ and $\overline{0}$ are the same term.

In Lean, the same computation would be a theorem: `normalize` with enough fuel, plus `normalize_sound`, proves that the factorial term reduces to `church 6`. The Rust program computes; the Lean development guarantees that the computation means what we think.

7.13 Common confusions

"**Substitution is search and replace.**" It is not, as the capture example shows: replacing text inside a binder can change which binder a variable refers to. Substitution is defined on syntax trees, respects scopes, and renames bound variables when necessary.

"**α-equivalent terms are different programs.**" They are the same program written with different parameter names. Every reasonable operation on terms gives α-equivalent results on α-equivalent inputs, which is why we identify them, and why de Bruijn indices, which make them equal, are so convenient.

"**A de Bruijn index is a position in the term.**" It counts *binders*, not symbols: it is the number of λs between the occurrence and its binder. The same variable can have different indices in different places.

"**Normal order is the efficient strategy.**" It is the *complete* one: it finds a normal form whenever there is one. It can be slower than call-by-value, because it may duplicate unevaluated arguments. Real languages choose call-by-value (Rust, OCaml) or call-by-need (Haskell).

"**Church–Rosser means every computation terminates.**" It means computations never disagree about the result. Ω is confluent and has no result.

"**The untyped λ-calculus can go wrong.**" Not in the sense of crashing: every term is a function, so every application can proceed. What it lacks is any protection against *meaningless* programs, such as adding a boolean to a pair; it computes an answer, just not a sensible one. Once we add genuine constants like numbers in Chapter 8, "applying the number 3" really does get stuck, and types become indispensable.

7.14 Summary

- The untyped λ-calculus has three constructs: variables, abstractions $\lambda x.\, t$ and applications $t\ u$. Functions of several arguments are curried.
- A λ binds its parameter in its body; the other variables are free. The free variables of a Rust closure's body are its captures.
- Substitution $t[x := s]$ must avoid capture: when a binder would capture a free variable of s, rename the binder first. Naive substitution changes the meaning of programs; `macro_rules!` hygiene is the same idea.

- β-reduction, $(\lambda x.\, t)\, s \to t[x := s]$, is the only computation rule. It duplicates and discards arguments freely.
- Strategies choose the redex: normal order (complete), call-by-name (lazy), call-by-value (Rust's). They can disagree on termination, never on the result: the Church–Rosser theorem makes normal forms unique.
- Ω diverges; Y and Z produce recursion from self-application; Church encodings represent booleans, pairs and numbers as functions, enough for all computable functions.
- De Bruijn indices replace names by binder distances: α-equivalence becomes equality, and capture avoidance becomes shifting.
- In Lean, terms are an inductive type, reduction is an inductive relation, a fuel-based normalizer is proved sound, and computation can produce proofs. Lean rejects a normalizer without fuel because it cannot terminate on every input; `partial` makes it runnable but opaque to proofs.
- Rust closures are typed λ-terms, and types rule out self-application: `|x| x(x)` has no type.

7.15 Exercises

1. Compute the free variables of $\lambda x.\, y\, (\lambda y.\, x\, y\, z)$, then compute $(\lambda x.\, y\, (\lambda y.\, x\, y\, z))[y := x\, z]$ by hand, step by step, renaming where necessary. Check your answer with the crate (`parse` and `Term::subst`, comparing with `alpha_eq`). *Hint:* the outer λx would capture the x of $x\, z$, so it must be renamed; the inner λy binds y itself, so the substitution stops there.
2. Reduce $(\lambda x.\, \lambda y.\, y\, x)\, ((\lambda z.\, z)\, a)\, (\lambda w.\, w)$ in normal order and in call-by-value, writing every step. Count the steps. Which strategy is faster here, and why? Confirm with `trace`. *Hint:* look at how many times x is used in the body.
3. Define Church encodings of `xor` and of $double = \lambda n.\, plus\ n\ n$ with the crate's `with` helper (copy it into the example, since it is private to the crate), and add rows for them to the table in `ch07_church.rs`. *Hint:* $xor = \lambda a.\, \lambda b.\, a\ (not\ b)\ b$.
4. Convert $\lambda f.\, (\lambda x.\, f\, (x\, x))\, (\lambda x.\, f\, (x\, x))$ (Y) to de Bruijn indices, then perform one step of β-reduction on $Y\, g$ (with g free, index 0 in the naming context $[g]$) using the definitions of shift and substitution, by hand. *Hint:* Y is $\lambda.\, (\lambda.\, 1\, (0\, 0))\, (\lambda.\, 1\, (0\, 0))$. The substitution enters one binder (the λx) on its way to each occurrence of f, so it looks for index 1 there and inserts g, shifted to index 1. The result looks exactly like the body of Y, but its index 1 now means g.
5. In Lean, prove `Steps (app I (app I I)) I` (with `I` from `Untyped.lean`) in two ways: by building the sequence with `Relation.ReflTransGen.head` and `Step` constructors, and by computation with `normalize_sound` and `decide`. Then state and prove that `I` is normal. *Hint:* for normality, `intro t h` and then `cases h` twice: a variable cannot step.
6. Prove in Lean that `normalize` is monotone in its fuel: if `normalize n t = some u` then `normalize (n + 1) t = some u`. *Hint:* induction on `n` generalizing `t`, and in the successor case `cases hs : step t` followed by `simp [normalize, hs]` on both the hypothesis and the goal.